SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 50**
Question Count: 5
Duration : 60 Minutes

Code of Conduct

I G. Naveen Kumar Reddy (192525288) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: *G. Naveen Kumar Reddy*

Assessment Tasks

Analytical Problem Solving

1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.
2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.
3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.
4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.
5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Equivalent Steps in VMware Workstation Pro

- Library pane showing multiple VMs = multiple instances/regions.
- VM → Snapshot → Take Snapshot, then Snapshot Manager = backup/versioning demonstration.
- Edit → Virtual Network Editor to show NAT/Host-only/Bridged networks = hybrid or multi-cloud connectivity.
- VM → Manage → Clone = replica/auto-scaling demonstration.

Tip: Paste each screenshot directly under the relevant question in this document (Insert → Picture in Word) before final submission, with a one-line caption explaining what it demonstrates.

Analytical Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 – Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Interpretation	20%	Interprets all technical requirements accurately	Interprets most requirements correctly	Partial interpretation	Misinterprets the problem
Analytical Reasoning	25%	Strong reasoning with clear cause-effect analysis	Good reasoning with minor gaps	Basic analysis only	Weak or no analysis
Method Selection	20%	Chooses highly appropriate virtualization and security concepts	Chooses mostly suitable concepts	Partially suitable concepts	Inappropriate approach
Solution Quality	20%	Provides feasible and technically sound solution	Reasonable solution with minor issues	Basic solution with limitations	Unclear or invalid solution
Presentation of Analysis	15%	Well-structured and technically precise	Generally clear	Somewhat unclear	Difficult to follow

Total Marks Awarded:

Q1. Type-1 vs Type-2 Hypervisors for a Mission-Critical Datacenter

Problem interpretation

The datacenter hosts mission-critical applications, so the hypervisor must deliver near-native performance, strong isolation, high availability and centralized, automated management at scale. Three criteria are required: performance, security and manageability, followed by a justified choice.

Comparison and analysis

Criterion	Type-1 (bare-metal): ESXi, KVM, Xen, Hyper-V	Type-2 (hosted): VMware Workstation, VirtualBox
Architecture	Hypervisor installed directly on server hardware and controls CPU, memory, I/O.	Runs as an application on a host OS (Windows/Linux), which controls the hardware.
Performance	Direct hardware access via VT-x/AMD-V, EPT/NPT, SR-IOV and paravirtual drivers. Typical overhead about 2–5%.	Extra layer: guest → hypervisor → host OS → hardware. Host OS processes compete for CPU/RAM. Typical overhead about 10–20%.
Security	Small, purpose-built code base (small attack surface), secure boot, TPM/vTPM, strong VM-to-VM isolation, fewer services to patch.	Large attack surface: the full host OS, its drivers and apps. Host malware or a host-OS exploit exposes every VM.
Manageability	Cluster-level management (vCenter, SCVMM, OpenStack): live migration, HA, DRS/load balancing, templates, APIs, automation.	Per-machine GUI management; no native clustering, HA or live migration; hard to automate at scale.
Availability / scale	HA restart, fault tolerance, hundreds of VMs per host.	Depends on host OS uptime and reboots; limited VM density.
Best fit	Production datacenters, cloud providers.	Developer test labs, training, desktop demos.

Calculations (assumed values)

- **Usable capacity:** host with 64 cores. Type-1 (4% overhead) gives 64 × 0.96 = **61.4 cores**; Type-2 (15% overhead) gives 64 × 0.85 = **54.4 cores**. Type-1 provides about **13% more usable capacity** (61.4 / 54.4 = 1.13).
- **Availability:** a Type-2 stack depends on both layers in series: A = A(hypervisor) × A(host OS) = 0.999 × 0.995 ≈ 99.4%, i.e. about 52.6 h downtime/year (0.006 × 8760). A Type-1 cluster with 2 hosts in HA: $1 - (0.001)^2 = 99.9999\%$, about 32 seconds/year.

Justification and conclusion

Type-1 is the better choice. It gives higher performance (lower overhead, more usable cores), a smaller and more hardened attack surface, and enterprise manageability (HA, live migration, automation) that mission-critical SLAs require. Type-2 remains suitable only for development, testing and training.

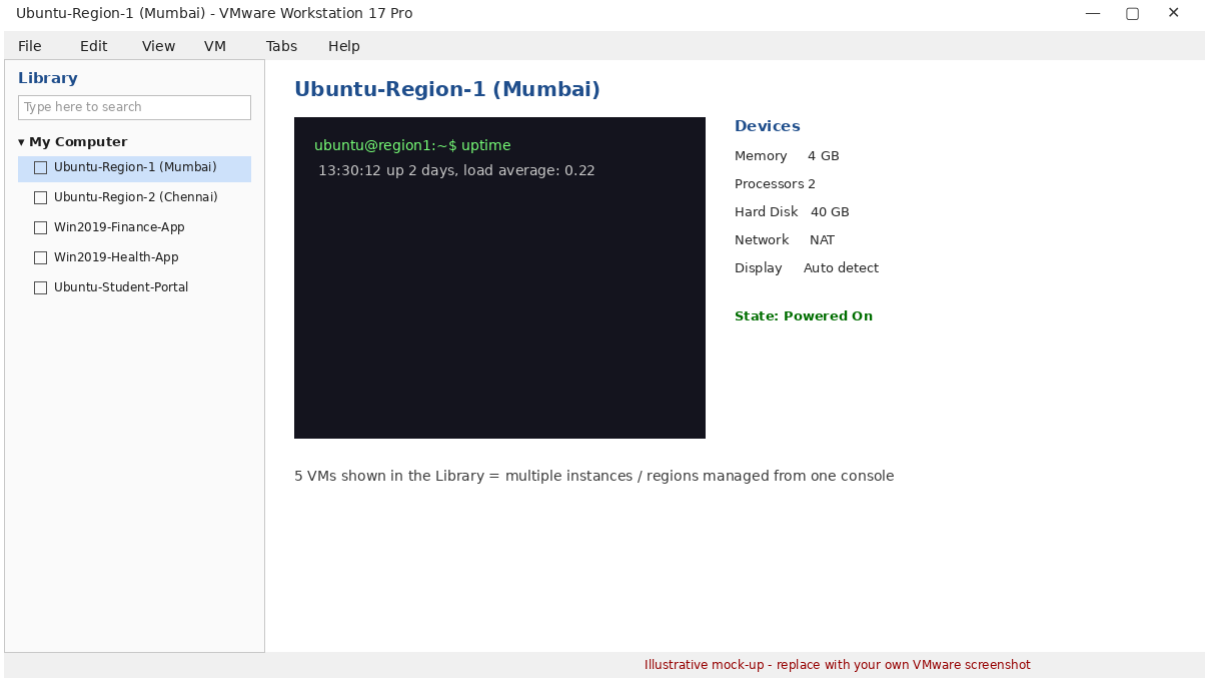


Figure 1: Library pane with multiple VMs, representing multiple instances/regions managed from one console.

Q2. Workload Isolation (Finance, Healthcare, Student Portal) and Attack Surfaces

Problem interpretation

Three tenants with different sensitivity and compliance needs (finance: PCI DSS/SOX; healthcare: HIPAA; student portal: FERPA/internet-facing) share the same physical hardware. The task is to show how the virtualization architecture enforces isolation and to name two major attack surfaces.

How virtualization provides isolation

Layer	Isolation mechanism	Tenant application
CPU / execution	Hardware-assisted virtualization (VT-x/AMD-V) with Ring -1 hypervisor mode; each vCPU runs in its own VM context; hypervisor scheduler enforces shares/limits.	Dedicated hosts or CPU reservations for finance and healthcare; no overcommit.
Memory	Second-level address translation (EPT/NPT) gives each VM a private guest-physical address space; memory zeroing on reallocation; page sharing disabled across tenants.	Memory reservations for regulated workloads; inter-VM page sharing off.
Storage	Separate virtual disks, datastores/LUNs with LUN masking, per-tenant encryption keys.	Encrypted, separate datastores for finance and healthcare.
Network	Virtual switches with VLAN/VXLAN segmentation, port groups, distributed firewall and micro-segmentation.	VLAN 10 finance, VLAN 20 healthcare, VLAN 30 student portal (DMZ); default-deny rules between them.
Management / policy	Role-based access control, resource pools, affinity/anti-affinity rules, audit logging.	Separate admin roles per tenant; anti-affinity keeps finance and student VMs on different hosts.

Two major attack surfaces

1. Hypervisor / VM-escape attack surface. The hypervisor is the single trusted layer under all tenants. Bugs in device emulation, virtual drivers or hypercalls (for example the VENOM floppy-controller flaw, CVE-2015-3456) can let a compromised student-portal VM break out and read or modify finance or healthcare VMs.

- Mitigation: apply hypervisor patches promptly, remove unused virtual devices, use a minimal Type-1 hypervisor, enable secure boot/TPM attestation, and place regulated tenants on dedicated hosts.

2. Shared-resource side-channel and co-residency attack surface. VMs share CPU caches, memory buses and network/storage I/O. An attacker co-located on the same host can use cache-timing or speculative-execution attacks (Spectre/Meltdown class) to infer secrets across VM boundaries, and a noisy neighbour can cause denial of service.

- Mitigation: dedicated hosts or core scheduling for sensitive tenants, disable hyper-threading for regulated hosts, apply microcode/side-channel patches, enforce resource limits, and use anti-affinity rules.

Conclusion

Hardware-assisted CPU/memory isolation, segmented virtual networks, encrypted storage and policy controls allow the three tenants to coexist safely, but the residual risk sits in the hypervisor and shared hardware, so the most sensitive tenants should additionally be placed on dedicated hosts.

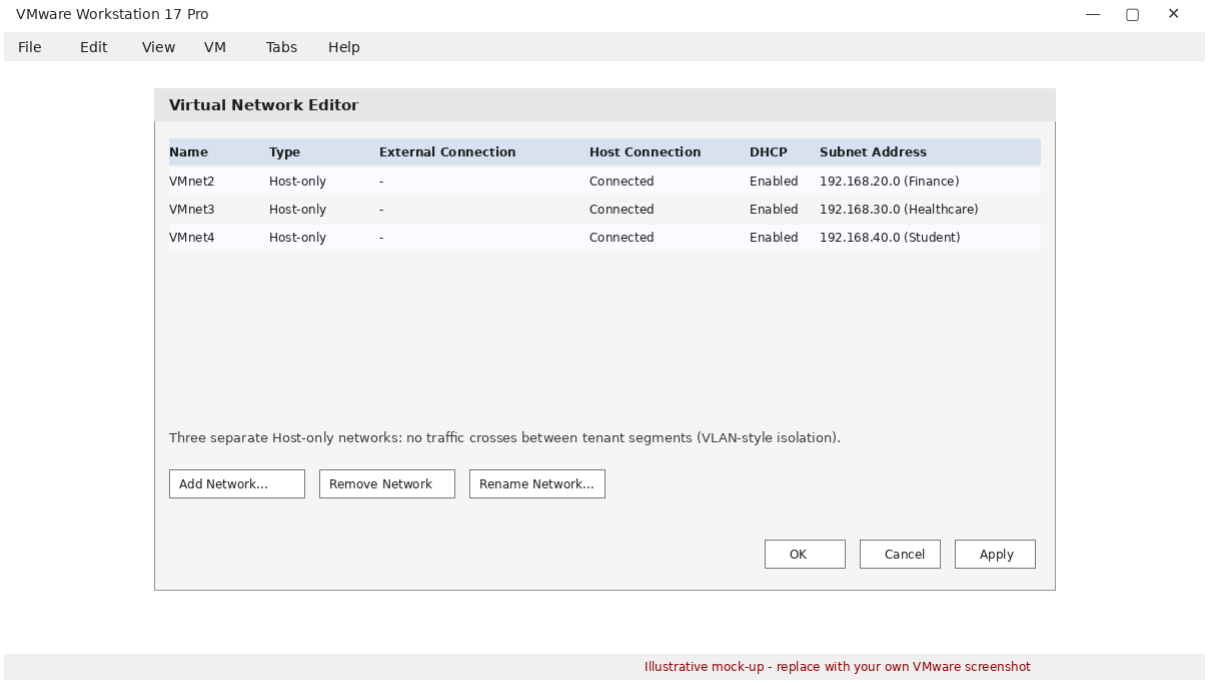


Figure 2: Virtual Network Editor with separate Host-only segments, demonstrating network isolation between tenants.

Q3. VM Host Utilization Snapshot: Bottlenecks and Corrective Actions

Problem interpretation

Observed: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. We must identify the bottlenecks and recommend corrective actions.

Analysis against healthy thresholds

Metric	Observed	Healthy target (typical)	Gap calculation	Status
CPU	92%	≤ 70–75% sustained	92 − 75 = 17 points over; only 8% headroom; 92/75 = 1.23× target	Critical (CPU contention)
Memory	88%	≤ 75–80%	88 − 80 = 8 points over; only 12% free for spikes	High (reclamation/ballooning risk)
Storage latency	35 ms	≤ 10–20 ms	35/20 = 1.75× ceiling; 35/10 = 3.5× ideal	Critical (primary bottleneck)
VM boot time	170 s	≈ 60 s	170/60 = 2.8×; 110 s excess	Poor (symptom)

Likely bottlenecks (cause-effect chain)

1. **Storage I/O is the primary bottleneck.** At 35 ms latency each VM is throttled: with one outstanding I/O, throughput = 1000 ms / 35 ms ≈ 29 IOPS, versus 1000 / 10 = 100 IOPS at 10 ms, i.e. 3.5× lower. Boot is I/O-heavy (OS image, drivers, services), so it slows directly.
2. **Memory pressure amplifies storage load.** With 88% memory used, the hypervisor starts ballooning, compression and host swapping, which adds extra disk I/O and further raises latency.
3. **CPU contention (92%) increases CPU ready time.** vCPUs wait for physical cores, so boot-time services and I/O completion handling are delayed.
4. **Probable trigger:** many VMs booting at the same time (boot storm) on shared storage and an over-consolidated host.

Chain: over-consolidation → memory pressure and CPU contention → extra swap I/O → latency 35 ms → boot 170 s.

Boot-time estimate (assumed: boot is 70% I/O-bound)

- I/O part = 0.70 × 170 = 119 s; other part = 51 s.
- Cutting latency from 35 ms to 10 ms: 119 × (10/35) ≈ 34 s. New boot ≈ 34 + 51 = **85 s**.
- Relieving CPU/memory contention (assume non-I/O part falls about 40%, 51 → ~31 s): boot ≈ 34 + 31 = ≈ **65 s**, close to the 60 s target.

Corrective actions

Time frame	Action
Immediate	Use vMotion/DRS to migrate VMs off the host; stagger VM power-on to avoid a boot storm; enable Storage I/O Control; stop or pause idle VMs; take a snapshot before changes.
Short term	Right-size VMs (reduce over-allocated vCPUs and RAM); enable memory compression/page sharing; move swap and boot images to SSD/NVMe tier; use linked/instant clones from a golden image so boots read a shared cached base.
Long term	Add capacity and set an N+1 headroom policy; alerting at CPU 75%, memory 80%, latency 20 ms; tiered storage (flash cache); capacity planning with monthly trending.

Capacity calculation (assumed 4-host cluster, all hosts equally loaded)

- CPU: hosts needed for 75% target = 4 × 92 / 75 = 4.91, so **5 hosts**; resulting CPU = 4 × 92 / 5 = 73.6%.
- Memory: 4 × 88 / 75 = 4.69, so 5 hosts; resulting memory = 4 × 88 / 5 = 70.4%.
- Adding **one host** brings both resources under the target. Right-sizing and migration reduce the need further.

Expected outcome: CPU ≈ 74%, memory ≈ 70%, latency ≤ 15 ms, boot time ≈ 60–65 s.

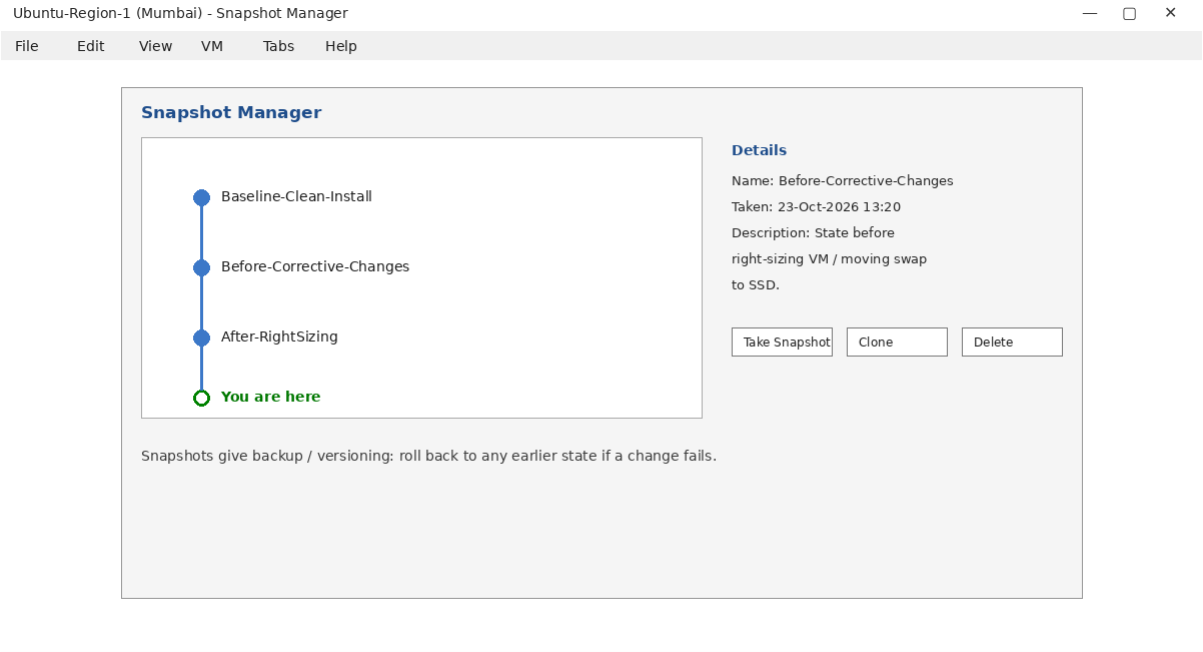


Figure 3: Snapshot Manager showing a snapshot taken before applying corrective changes (backup/versioning).

Q4. Software Defined Datacenter (SDDC) vs Traditional Datacenter: Agility

Problem interpretation

Agility means how fast and cheaply infrastructure can be provisioned, changed, scaled and recovered. We compare SDDC, where compute, storage, network and security are virtualized and policy/API-driven, with a traditional datacenter built from hardware-defined silos.

Comparison

Factor	Traditional datacenter	Software Defined Datacenter
Provisioning time	Procure, rack, cable, configure VLANs: days to weeks.	Template/API/self-service portal: minutes.
Resource utilization	Dedicated servers, typically 10–20% used.	Pooled resources, typically 60–80% used.
Change and scaling	Manual, ticket-based; vertical scaling needs downtime.	Automated, elastic scale-out/in, policy-based; infrastructure as code.
Recovery	Hardware-specific backup/DR, slow restore.	Snapshots, replication, VM restart on any host, runbook automation.
Management	Separate tools per silo (server, SAN, network).	Single control plane and common APIs.
Cost model	High CapEx, over-provisioned for peak.	Commodity hardware, pay for what is used (OpEx-like).

Virtualization considerations

- **Compute virtualization:** the hypervisor abstracts CPU/RAM into pools; live migration (vMotion), DRS and HA enable load balancing and fast recovery; templates and clones give fast VM creation and auto-scaling.
- **Storage virtualization:** software-defined storage (vSAN, Ceph) pools local disks into one datastore; policy-based provisioning (RAID level, IOPS, encryption per VM), thin provisioning, snapshots and tiering remove manual LUN work.
- **Network virtualization:** overlays (VXLAN) and virtual switches/routers (NSX-style) let networks, firewalls and load balancers be created in software in seconds; micro-segmentation applies security per workload and follows the VM when it moves.

Calculations (assumed values)

- **Provisioning speed:** about 2 weeks (14 × 24 × 60 = 20,160 min) in a traditional datacenter vs about 15 min in SDDC gives ≈ **1,300× faster** (20,160 / 15 = 1,344).
- **Consolidation:** 100 physical servers at 15% average utilization carry 100 × 15 = 1,500 utilization points. At a 70% target, hosts needed = 1,500 / 70 = 21.4, so **22 hosts**, a hardware reduction of 78%, with the freed capacity available to absorb new workloads without new purchases.

Challenges and conclusion

Considerations: overlay overhead, licensing, staff skills and the security of the central control plane (which needs RBAC, hardening and audit). Overall, SDDC is far more agile because abstraction plus automation turns infrastructure changes from hardware projects into software operations.

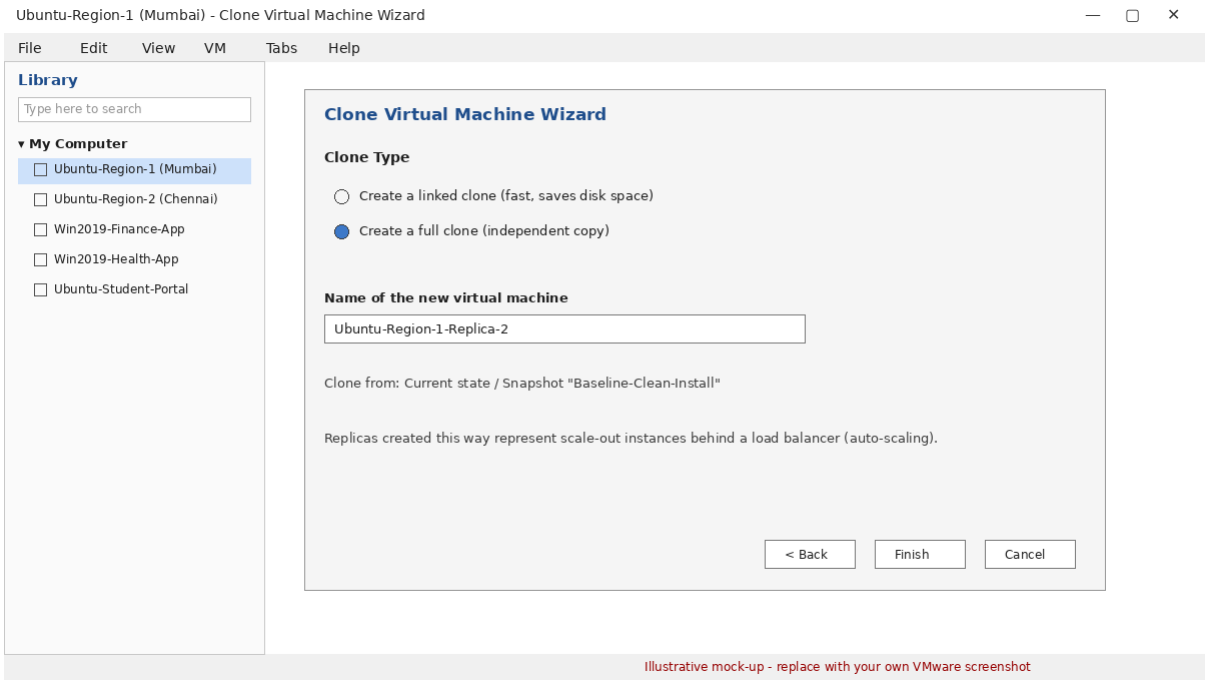


Figure 4: Cloning a VM to create replicas, demonstrating rapid provisioning and auto-scaling.

Q5. Multi-Cloud Federation Failure: Identity Mismatch and Secure Design

Problem interpretation

Each cloud provider has its own identity system (for example AWS IAM, Azure/Entra ID, Google Cloud IAM). Without a common trust model, a user known in one provider is not recognised in another, so federation fails. We must find the mismatches and design a secure, privacy-preserving federated identity solution.

Root-cause analysis

Mismatch	Why it breaks federation	Fix
Different identity stores and no shared trust	Each provider only trusts its own directory; no exchange of IdP metadata or signing certificates.	Establish explicit trust between IdP and service providers using metadata exchange.
Protocol differences (SAML 2.0, OIDC/OAuth 2.0, Kerberos, proprietary tokens)	A token issued in one format is rejected by a provider expecting another.	Standardise on SAML 2.0 and OIDC; use a broker/gateway to translate.
Attribute and identifier mismatch (email vs UPN vs ARN; different role names)	Claims cannot be mapped to local accounts or roles, causing authentication or authorization failure.	Canonical attribute schema and claim-mapping rules; map groups to roles.
Inconsistent authorization models (IAM roles vs RBAC vs policies)	Over-privileged or denied access.	Role mapping with least privilege; attribute-based access control (ABAC).

Mismatch	Why it breaks federation	Fix
Weak lifecycle and assurance (no deprovisioning, different MFA levels, clock skew)	Orphaned accounts, assertion rejection (token validity windows).	SCIM provisioning/deprovisioning, uniform MFA policy, NTP time sync.

Proposed secure identity and privacy design

Federation concepts used: Identity Provider (IdP), Service Provider (SP) / relying party, single sign-on (SSO), trust framework, claims-based tokens and identity brokering.

1. **Hub-and-spoke identity broker:** one trusted enterprise IdP (or federation broker) is the single source of truth; each cloud (AWS, Azure, GCP) is an SP that trusts the IdP via SAML 2.0/OIDC metadata and signed assertions.
2. **Authentication:** SSO with MFA (step-up MFA for privileged roles); short-lived tokens (about 15–60 min) and signed, encrypted assertions over TLS 1.2+; regular signing-key rotation.
3. **Authorization:** canonical claims (user ID, group, department, assurance level); claim-to-role mapping at each provider; least privilege with ABAC; temporary credentials via STS/workload identity federation instead of long-term keys.
4. **Provisioning:** SCIM for automatic create/update/deprovision so leavers lose access in all clouds quickly; periodic access reviews.
5. **Privacy:** data minimisation (release only needed attributes), pairwise pseudonymous identifiers (a different ID per provider so users cannot be correlated across clouds), user consent and purpose limitation, encryption of attributes, and alignment with GDPR and India DPDP Act 2023.
6. **Monitoring:** centralised audit logs of authentications and token issuance, anomaly detection, and incident response/revocation.

Calculation: why a broker scales better

Point-to-point trust between N providers needs $N(N-1)/2$ relationships; a hub needs N. For N = 5: $5\times4/2 =$ **10** trusts vs **5** with the hub (50% fewer). For N = 10: 45 vs 10 (78% fewer), so fewer certificates, mappings and misconfigurations to maintain.

Conclusion

The failure comes from missing trust, protocol and attribute mismatches and weak lifecycle control. A brokered SAML/OIDC federation with canonical claims, MFA, short-lived tokens, SCIM and privacy-preserving pseudonymous identifiers resolves the mismatch while protecting user data.

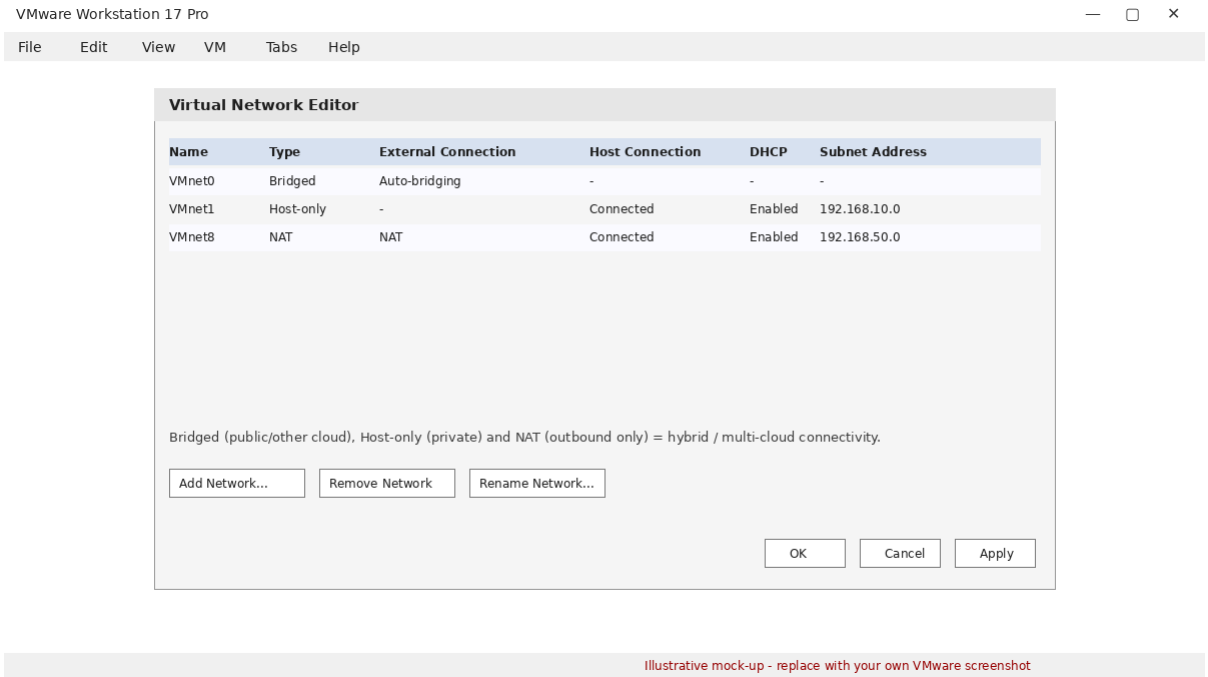


Figure 5: Virtual Network Editor with NAT, Host-only and Bridged networks, representing hybrid/multi-cloud connectivity.